# Scoping Microservices Right: Loose Coupling & High Cohesion

*Domain-Driven Design (DDD), Bounded Contexts, Connascence, and Avoiding the Distributed Monolith Trap*

**THE SCOPING PARADOX**

The term 'micro' is fundamentally misleading. Microservices are not defined by lines of code, team headcount, or function counts. Rather, microservices are defined by **bounded semantic autonomy**. Improperly scoped services result in a 'Distributed Monolith': the operational complexity of distributed systems coupled with the release locks of a monolithic codebase.

## 1. THE TWO PILLARS: LOOSE COUPLING & HIGH COHESION

Architectural quality in distributed systems is governed by the dynamic equilibrium between coupling and cohesion:

**Loose Coupling (Inter-Service Independence)**

Change in Service A requires zero coordinated change in Service B.

Minimizes blast radius • Enables independent deployment pipelines

**Goal: Asynchronous, event-driven, backwards-compatible contracts**

**High Cohesion (Intra-Service Affinity)**

Things that change together for the same business reason stay together.

Single domain authority • Consistent transactional integrity

**Goal: Complete functional autonomy within a Bounded Context**

## 2. TAXONOMY OF COUPLING IN DISTRIBUTED ARCHITECTURES

**1. Schema / Data Coupling** Critical

Two services reading/writing to the same shared database tables. Schema migrations break both services simultaneously, preventing independent releases.

**2. Temporal / Execution Coupling** High

Service A calls Service B synchronously; Service B calls Service C. If Service C is slow or down, Service A fails. Availability compounds multiplicatively: = A_A imes A_B imes A_C$.

**3. Contract / Interface Coupling** Moderate

Client tightly depends on internal data representations. Mitigated by consumer-driven contracts, protocol buffers with field tags, and strict additive evolutions.

## 3. DEGREES OF COHESION: FROM ANTI-PATTERN TO OPTIMAL

| Cohesion Type | Architectural Manifestation | Impact & Evaluation |
|---|---|---|
| **Functional Cohesion** | All operations execute a single well-defined domain capability (e.g. Invoicing Service). | Ideal High reusability, clear invariants, minimal external chatty traffic. |
| **Sequential Cohesion** | Output of operation A feeds directly into operation B (e.g. Order Processing Pipeline). | Strong Natural streaming or event-driven pipeline candidate. |
| **Temporal Cohesion** | Operations grouped only because they happen at the same time (e.g. Nightly Cron Service). | Fragile Combines unrelated business domains; causes cascade failures. |
| **Coincidental Cohesion** | Operations grouped arbitrarily (e.g. 'CommonService' or 'UtilService'). | Anti-pattern Highest coupling; changes cause ripple effects across all teams. |

## 4. DOMAIN-DRIVEN DESIGN (DDD) & BOUNDED CONTEXTS

Domain-Driven Design provides the mathematical and conceptual rigor required to discover optimal service boundaries. A **Bounded Context** defines the linguistic and semantic perimeter within which a domain model applies without ambiguity.

### The Polysemic Concept: The 'User' Entity Across Bounded Contexts

In a monolith, a single bloated `User` table often contains 80 columns mixing auth passwords, shipping addresses, payment tokens, and marketing engagement scores. In DDD, each Bounded Context owns an isolated model tailored to its domain:

#### Identity Context

```
User {
  userId: UUID,
  email: String,
  passwordHash: String,
  mfaSecret: String
}
```

#### Shipping Context

```
Recipient {
  userId: UUID,
  shippingAddress: Address,
  deliveryInstructions: Text,
  contactPhone: String
}
```

#### Billing Context

```
Customer {
  userId: UUID,
  paymentMethodToken: String,
  billingAddress: Address,
  taxId: String
}
```

## 5. CONTEXT MAPPING PATTERNS: MANAGING CROSS-BOUNDARY RELATIONSHIPS

| Relationship Pattern | Mechanism & Implementation | Coupling Characteristic |
|---|---|---|
| **Shared Kernel** | Two contexts share a subset of the domain model and database. Both teams must coordinate on every change. | High Coupling Use sparingly; risks distributed lock-in. |
| **Customer / Supplier** | Upstream service supplies data; Downstream context requests features and schemas to fit consumption needs. | Directed Coupling Upstream roadmap must accommodate downstream SLAs. |
| **Anti-Corruption Layer (ACL)** | Downstream service implements an adapting translation layer to isolate its clean domain model from legacy upstream schemas. | Loose Coupling Zero upstream leakage; protects domain integrity. |
| **Separate Ways** | Contexts duplicate minimal logic or data independently rather than incurring cross-network RPC dependencies. | Zero Coupling Optimal operational autonomy at the cost of storage. |

## 6. MATHEMATICAL COUPLING METRICS (ROBERT C. MARTIN FORMALISM)

We quantify service coupling using package metrics adapted to distributed microservice boundaries:

```
Afferent Coupling (Ca): Number of external services depending on this service (Incoming calls).
Efferent Coupling (Ce): Number of external services this service depends on (Outgoing calls).

Instability Metric (I):
  I = Ce / (Ca + Ce)    where I in [0, 1]
  * I = 0  --> Maximally Stable (Autonomous service, e.g. Core Auth). Hard to change because many depend on it.
  * I = 1  --> Maximally Unstable (Leaf orchestrator, e.g. API Aggregator). Easy to change, depends on many.

Stable Abstractions Principle:
  A service should be as abstract as it is stable. Services with low I must maintain immutable, versioned contracts.
```

## 7. THE SCOPING DECISION MATRIX: WHEN TO SPLIT VS. MERGE

Architectural boundaries should be adjusted dynamically based on telemetry and team organizational realities:

### Indicators to Split into Independent Services

- **Independent Scaling Vector:** Service component requires 50x more memory or GPU compute than neighbors.
- **Divergent Change Frequencies:** Feature branch A deploys 30 times a day; feature branch B requires regulatory audit twice a year.
- **Different Failure Domains:** A crash in non-critical recommendation engine must not take down the checkout flow.
- **Team Ownership Alignment:** Two independent teams are editing the same codebase and suffering merge conflicts.

### Indicators to Merge (Distributed Monolith Red Flags)

- **Chatty RPC Cascades:** Every single business transaction requires 5–8 synchronous roundtrips between Service A and B.
- **Lockstep Deployments:** Service A cannot be deployed to production without simultaneously deploying Service B.
- **Distributed 2-Phase Commits:** Frequent requirement for distributed transactions (XA / 2PC) to maintain consistency.
- **Single Team Overhead:** A team of 3 developers maintains 12 microservices, spending 70% of time on pipeline maintenance.

## 8. DECLARATIVE ANTI-CORRUPTION LAYER (ACL) ARCHITECTURE

```
// Go Implementation: Anti-Corruption Layer isolating modern domain from legacy ERP
type LegacyOrderResponse struct {
    TX_ID       string
    CUST_NUM    int
    AMT_CENTS   int64
    STAT_CD     string  // e.g. "01" = Pending, "02" = Cleared
}

func (acl *ERPAdapter) TranslateOrder(ctx context.Context, orderID string) (*domain.Order, error) {
    raw, err := acl.erpClient.GetRawTransaction(ctx, orderID)
    if err != nil {
        return nil, fmt.Errorf("upstream ERP error: %w", err)
    }
    // Clean Domain Translation: Shields core domain from archaic external identifiers
    return &domain.Order{
        ID:         domain.OrderID(raw.TX_ID),
        CustomerID: domain.CustomerID(fmt.Sprintf("CUST-%d", raw.CUST_NUM)),
        Amount:     money.NewFromCents(raw.AMT_CENTS, money.USD),
        Status:     mapERPStatus(raw.STAT_CD),
    }, nil
}
```

## 9. PRODUCTION SCOPING CHECKLIST

### Architectural Verification Gates

1. **Shared Nothing Datastores:** Ensure each service has its own private logical and physical datastore instance.
2. **Ubiquitous Language Audit:** Check whether terms have identical definitions across domains; if they diverge, create a bounded context boundary.
3. **Autonomous Deployment Test:** Can Service A deploy to production at 2 PM on Friday while Service B is running an older version? If no, coupling is excessive.
4. **Network Boundary Justification:** Does replacing in-memory function calls with network RPCs provide proportional organizational or scalability benefits?